Advanced Graphics Programming in Unreal, part 4

25 min read · Jun 25, 2025

William Mishra-Manning

This is a seven-part article series.

Part 1: Introduction
Part 2: Rendering Abstractions (RHI, RDG) and Threading
Part 3: Scenes. Views, and SceneViewExtension
Part 4: Global Shaders
Part 5: Simple Material Shaders
Part 6: Advanced Material Shaders
Part 7: Mesh-Material Shaders and Mesh Passes

Finally it's time for shaders! This article covers the simplest type of Unreal shader, while the next 3 get increasingly complex. This is also the last article in the series to contain a lot of information which can be found elsewhere on the internet.

Global Shaders

If you read any other resources on custom Unreal shaders, then you already know about Global Shaders. They're pure text HLSL which come with an associated C++ class and need to be invoked by C++ code. They're immediately compiled on engine startup, and can be recompiled within the editor by pressing "***Ctrl+Shift+.***".

This article will go over them in more detail than any other resource I've found. It's the longest article in the series, but sets the stage for the next 3. The **Theory** and especially **Details** sections are more of a reference than a tutorial, so you may want to start with the **Demo** section.

In subsequent articles we'll upgrade to *Material Shaders*, which compile against a user Material; and add *Mesh-Material Shaders*, which compile against a Material **and** specific 3D primitive type.

Theory

All 3 shader types in Unreal (`FGlobalShader` , `FMaterialShader` and `FMeshMaterialShader`) inherit from `FShader` and share much of the same basic architecture. They do almost everything through **shadowing** — defining new functions, fields, or typedefs that hide the default ones inherited from `FShader` . This is one C++ alternative to object-oriented tricks like virtual functions. It works as long as the shader's precise type is known at compile time, which is why many functions interacting with shaders are templated. `FShader` and its 3 child classes are an important reference for writing new shaders of any kind.

Global Shaders are the most straightforward. They have a specific type (Vertex, Fragment, Compute, etc.) which Unreal calls the *frequency*. They are implemented in an HLSL text file (Unreal uses the extension .ush for headers and .usf for implementations). Finally, they are connected to Unreal with a C++ class inheriting from `FGlobalShader` .

Global shaders are pretty easy to invoke too! You can dispatch it from literally anywhere with the help of `ENQUEUE_RENDER_COMMAND()` .

Permutations

Global Shaders have multiple compiled variants, primarily based on your platform capabilities (SM5 vs SM6 vs ES3_1) and the current Quality Setting. However they can also define *permutations*. These are enum values, integer values, or bool flags that get fed into your shader as compile-time constants.

If your shader needs permutations then refer to the **Details>Permutations** section at

the bottom of this article for implementation details.

Shader Maps

A global *Shader Map* stores the set of all compiled variants of all global shaders, at **one** feature level. In other words there is one global shader map per feature level, which you can get with the function

```
GetGlobalShaderMap(level)
```

There is also a global variable for your machine's max feature level, `GMaxRHIFeatureLevel`, leading to the tempting snippet

```
GetGlobalShaderMap(GMaxRHIFeatureLevel)
```

However in most cases you should pull the feature level from the view you are rendering to. In fact, views offer a convenient handle to the global shader map you should use when rendering to them: `view.ShaderMap`. here is the full snippet to retrieve the shader for a viewport, assuming you have no permutations (otherwise see **Details>Permutations** below):

```
TShaderMapRef<FMyShader> myShader{ view.ShaderMap };
```

HLSL

To connect your shader class to an HLSL file implementing it, do the following:

1. Put your shader code in a *.usf* file, within the shader folder we set up in part 1.
2. Inside the shader class, invoke the macro `DECLARE_EXPORTED_SHADER_TYPE(T, Global, )`, where T is your class. In the future when we define Material and

Mesh-Material shaders, the second argument will be replaced with `Material` or `MeshMaterial` respectively. The third argument is an optional dll export token (like `MYMODULE_API`), if you want the class to be directly usable outside its module. – There are several variants of this macro you can use instead which provide minor conveniences, like `DECLARE_EXPORTED_GLOBAL_SHADER` or `DECLARE_SHADER_TYPE`.

3. In a cpp file, invoke this macro:

```
IMPLEMENT_SHADER_TYPE(templateDecl, Class, VirtualShaderPath, MainFunction, Fre
//For example:
IMPLEMENT_SHADER_TYPE(, FMyNewVS, TEXT("/Plugin/MyPlugin/myShader.usf"), TEXT("
```

– There are several variants of this macro you can use instead which provide minor conveniences, like `IMPLEMENT_GLOBAL_SHADER` or `IMPLEMENT_MATERIAL_SHADER_TYPE`.

4. The shader will be compiled as Unreal starts up, and any compilation errors will halt the startup process.

Headaches

Important Notes!!! *Unreal's shader parser has a few bugs which will leave you banging your head against a wall for a few days if you don't know about them.*

Never use the explicit `uniform` keyword when declaring a parameter in HLSL, or UE's shader compiler process will crash. `uniform` is already the default behavior when no keyword is provided, and you should always rely on that. Fortunately the other HLSL keywords like *static* work as expected.

Never declare multiple uniforms in one statement. If you write `float A, B;` then Unreal will only see `A` and ignore `B`.

Never add underscores to the front of your uniforms. This breaks the parser, we believe because it appends its own underscore and some shader compilers reserve double-underscored variables for internal use. This habit may be hard to break for

those of you coming from Unity.

As mentioned in the Debugging section of the first article, **DX12 will throw a very generic "invalid pipeline state" error if your vertex shader outputs don't line up perfectly with your pixel shader inputs.** Be careful when using your own pixel shader with a built-in vertex shader!

Parameters

Runtime parameters for your shader are normally defined with *parameter structs*.

1. Use a special set of macros to define a struct whose fields are uploadable shader uniforms. This starts with `BEGIN_SHADER_PARAMETER_STRUCT(structName, dllExportToken)` and ends with `END_SHADER_PARAMETER_STRUCT()`.
2. Inside your shader class, shadow the inherited typedef `FParameters` to point to that struct: `using FParameters= structName;`
3. Also inside your shader class, invoke the macro `SHADER_USE_PARAMETER_STRUCT(FMyShaderClass, FGlobalShader)`.

Within a parameter struct, you can add many kinds of data. Below is, to my knowledge, an exhaustive list of examples.

```
BEGIN_SHADER_PARAMETER_STRUCT(FMyShaderParams, MY_MODULE_API)

//Add a single matrix.
//Must have in HLSL: float4x4 MyMatrix;
SHADER_PARAMETER(FMatrix44f, MyMatrix)
//Add an array of vectors.
//Must have in HLSL: uint4 MyUvecs[2];
SHADER_PARAMETER_ARRAY(FUintVector4, MyUvecs, [2])

//Add Sampleable textures and read-only buffers, a.k.a.
//   "Shader Resource Views", or SRV's.
// * Here's a texture assuming you use this shader in an RDG pass,
//     requiring the HLSL: Texture2D<float2> MyTex;
SHADER_PARAMETER_RDG_TEXTURE_SRV(Texture2D<float2>, MyTex)
// * Here's a buffer assuming you use this shader outside an RDG pass
//     (or the buffer is always read-only so memory barriers don't matter),
//     requiring the HLSL: Buffer<uint2> MyUvecs2;
```

```
SHADER_PARAMETER_SRV(Buffer<uint2>, MyUvecs2)

//Add nonsampleable read-write textures, and read-write buffers, a.k.a.
//   "Unordered Access Views", or UAV's.
// * If using an RDG pass you MUST declare them with RDG handles, like this
//     requiring the HLSL: RWTexture2D<float4> MyOutputTex;
SHADER_PARAMETER_RDG_TEXTURE_UAV(RWTexture2D<float4>, MyOutputTex)
// * If not using an RDG pass then you can store them as a plain RHI handle,
//     like this, requiring the HLSL: RWBuffer<float> MyOutputFloats;
SHADER_PARAMETER_UAV(RWBuffer<float> MyOutputFloats;

//Add sampler objects.
//   Requires the HLSL: SamplerState MySampler;
SHADER_PARAMETER_SAMPLER(SamplerState, MySampler)
//   Requires the HLSL: SamplerState MySamplers_0;
//                      SamplerState MySamplers_1;
//                      SamplerState MySamplers_2;
SHADER_PARAMETER_SAMPLER_ARRAY(SamplerState, MySamplers, [3])

//Add a reference to a global buffer.
//The struct referenced here is defined in VIEW_UNIFORM_BUFFER_MEMBER_TABLE,
//   partway down Engine/Public/SceneView.h, line 689 as of Unreal 5.3.
SHADER_PARAMETER_STRUCT_REF(FViewUniformShaderParameters, View)
// Example HLSL usage: float4x4 worldToView = View.TranslatedWorldToView;
// To set this parameter in the struct you provide a handle to the global buffe
//     for example given your current View you can do:
//     params->View = currentView.ViewUniformBuffer;

//Paste another parameter struct into this one.
//The inner struct exists in C++, but
//    in HLSL the nested fields live next to our other ones.
SHADER_PARAMETER_STRUCT_INCLUDE(FNestedParamStruct, MyCppName)

//Nest another parameter struct within this one.
//The fields are nested both in C++, and in HLSL using a name prefix.
//This requires HLSL declarations of each field of the nested struct:
//   float NestedData_MyFloat; int NestedData_MyInts[2]; etc
SHADER_PARAMETER_STRUCT(FNestedParamStruct, NestedData)

//Add an array of some other parameter struct.
//This works similar to SHADER_PARAMETER_STRUCT, but also adding
//    the array index to the prefixed name of each field.
//This example requires HLSL declarations like so:
//    float MyStructs_0_MyFloat; int MyStructs_0_MyInts[2];
//    float MyStructs_1_MyFloat; int MyStructs_1_MyInts[2];
//    float MyStructs_2_MyFloat; int MyStructs_2_MyInts[2];
SHADER_PARAMETER_STRUCT_ARRAY(FNestedParamStruct, MyStructs, [3])
```

```
END_SHADER_PARAMETER_STRUCT()
```

As mentioned above, most of the time you're using an RDG to execute your shader and you should use the RDG declaration of textures/buffers rather than the standard RHI ones. When declared this way the parameter struct's fields are an RDG handle rather than an RHI handle, allowing you to bind RDG temporary resources to them. This also tells the render graph that your pass uses those objects, which is especially important for objects that are written to in some passes and sampled from in others. The RDG issues memory barriers for all resources it knows about.

Parameter Uploading

To send a parameter struct to your shader, do one of the following:

- If working with a plain RHI command-list (usually not the case but you still need to know this), you can create the parameter struct anywhere, such as a local variable. Set its fields, then call the global helper function `SetShaderParameters(cmds, shaderRef, shaderRef.GetPixelShader(), params)`. Replace `GetPixelShader()` with your own shader frequency.
- If working with a higher-level RDG (usually the case), you should allocate the parameters within the graph using `graph.AllocParameters<FMyParamStruct>()`. This ensures that the parameter struct lives for the duration of the graph's execution. Then you need to wait until execution time to call `SetShaderParameters(…)`, meaning you should call it within your `AddPass(…)` lambda. Luckily, in practice you will usually call global helper functions that wrap `AddPass`, such as `AddDrawScreenPass()`, and those usually call `SetShaderParameters(…)` for you. You also need to pass the struct, or an outer one that owns it, into `AddPass(…)` itself.

In some cases your parameter struct will not be able to list *all* parameters for your shader. You may have parameters bound from external sources (Material shaders, covered in the next article, must generate and bind parameters from their Material), or you may inherit from a shader class which uses Legacy parameters (described below). Regardless of the reason, you need to tell Unreal to not freak out when your

parameter struct is missing some uniforms, by replacing `SHADER_USE_PARAMETER_STRUCT(…)` with `SHADER_USE_PARAMETER_STRUCT_WITH_LEGACY_BASE(…)`. If you forget to do this you will get an assert failure, however you can skip over it in the debugger and nothing bad should happen.

Legacy Parameters

There is an important "Legacy" alternative to parameter structs. It shows up in various engine shaders, and is *required* for Mesh-Material shaders. It involves declaring individual fields for each parameter within the shader class, then binding them manually in two different places.

We won't need them until the article on Mesh-Material shaders, but it's all over the engine code and I've described them below in the **Details** section.

Workflow

It's common to hide the entire shader in a cpp file, and only expose a global function which invokes that shader the correct way. This is a form of encapsulation, making it impossible to misuse your shader. This convention also applies to Material and Mesh-Material shaders covered in later articles.

If you wish to expose your shader directly then make sure to declare the class in a header, and provide your DLL export token (e.x. `MYMODULE_API`) when invoking `DECLARE_EXPORTED_SHADER_TYPE(…)`. Otherwise your users will get linker errors.

If you want to write a Vertex and Pixel shader pair that's closely tied to each other, you can keep them in the same USF file with separate entry points (usually named `MainVS` and `MainPS`). In fact, any number of shader classes can reference the same USF file. There are built-in preprocessor flags telling your shader what frequency is compiling: `VERTEXSHADER`, `PIXELSHADER`, `COMPUTESHADER`, etc. You can also have your shader classes inject specific defines into the compilation environment by shadowing the inherited static function `ModifyCompilationEnvironment(…)`.

Built-in Parameters

Many shaders throughout the engine reference data about the current viewport or current scene. A previous article mentioned how to add that into your own shader,

but we will restate it here.

View data can be referenced by adding `SHADER_PARAMETER_STRUCT_REF(FViewUniformShaderParameters, View)`, and extracted from an *`FViewInfo`* with `view.ViewUniformBuffer`. The data provided by this structure can be found in the definition of `VIEW_UNIFORM_BUFFER_MEMBER_TABLE`, partway down *Engine/Public/SceneView.h*, line 689 as of Unreal 5.3. You can access the view data in your shader through the global View, however it's recommended to invoke `ResolvedView = ResolveView();` at the start of your shader and then reference the global `ResolvedView` instead. This will automatically handle stereo rendering cases (VR) for you, and nearly every Unreal shader starts with that statement.

Scene data can be referenced by adding `SHADER_PARAMETER_RDG_UNIFORM_BUFFER(FSceneUniformParameters, Scene)`, and extracted from an `FViewInfo` with `view.GetSceneUniforms().GetBuffer(rdgInstance)`.

G-buffer Textures are provided to you if you hook into the right place in the render pipeline (see Scene View Extensions in part 3), and you can add them to your shader parameter struct with `SHADER_PARAMETER_RDG_UNIFORM_BUFFER(FSceneTextureShaderParameters, SceneTextures)`.

Demo

We are finally going to start on the effect described in the first article, by adding four global shaders to the project: one to initialize the Game of Life sim, one to resample the sim state when the player's resolution/screen percentage changes, one to tick the sim, and one to display the sim state on the screen like a post-process effect.

These shaders will offer a few parameters to customize their behavior, which will be fed in from the controlling subsystem. In the next article we will remove the need to manually pass a bunch of parameters, by switching the three main shaders from Global to Material so that most parameters and logic will come from the Material itself.

The Game of Life is a discrete simulation, where each cell (pixel) is either alive or

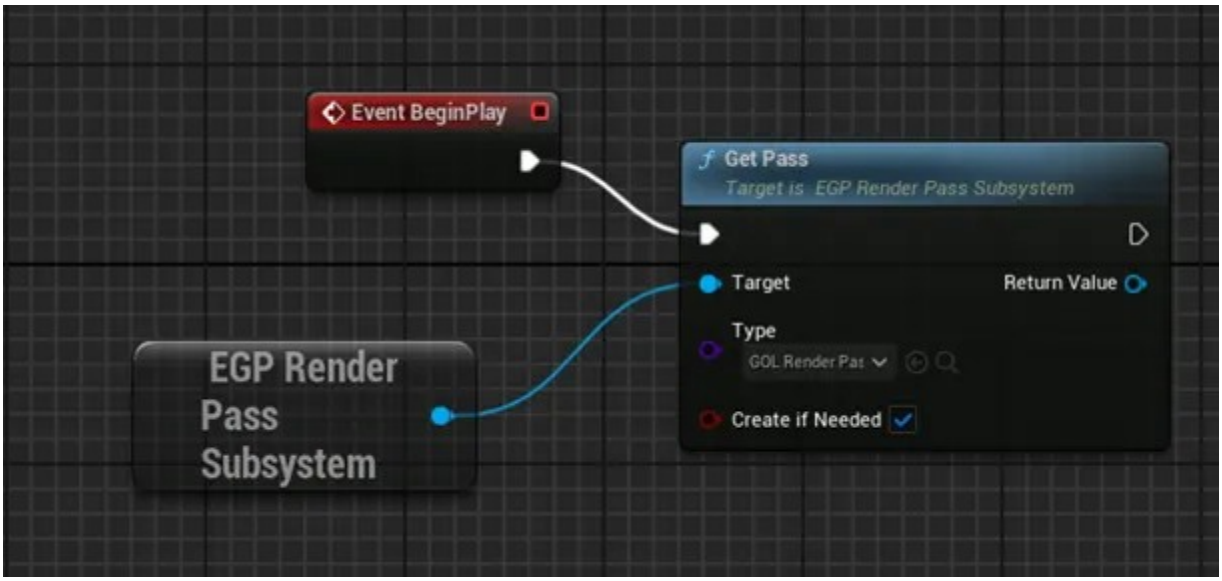
dead. We can represent this easily enough with a one-channel texture whose pixels are either 0 or 1. However to provide more interesting display options we'll add a second channel whose values are continuous, constantly animating towards that binary value.

I also added a special actor to the demo scene which renders and displays a Scene Capture Component. This demonstrates that the effect automatically runs in every viewport like any other part of the renderer.

William Manning

Watch on

As mentioned in the previous article, you need to make sure the pass gets created during gameplay. I do this in the level blueprint.



Same as before, but without the stuff about randomly clearing render targets

Sim State Data

Each viewport will have its own Game of Life sim, so we need to manage persistent per-view data. Unreal has no easy way to know when a viewport is created or especially deleted, but my library has a class to deal with that for you. It automatically cleans up a viewport's data after that viewport hasn't been used for a certain number of frames. You can also mark a viewport to prevent automatic cleanup no matter how infrequently it's used.

```
//In the header:
struct GOL_DEMO_API FGameOfLifeView final : public F_EGP_ViewPersistentData
{
    static FRHITextureCreateDesc SimStateDesc(const FInt32Point& viewportSize);

    //The simulation ping-pongs between two textures.
    TRefCountPtr<FRHITexture2D> SimState, SimBuffer;

    FGameOfLifeView(FRDGBuilder& graph, const FViewInfo& view,
                    const FIntRect& viewportSubset);
    //Move constructor/assignment and destructor are handled automatically,
    //    thanks to the textures' ref-counted pointer.
};

//In the cpp:
FRHITextureCreateDesc FGameOfLifeView::SimStateDesc(const FInt32Point& viewport
```

```
{
    //The sim looks pretty nice running at half-resolution.
    //This also cuts shader costs by 75%.
    auto texSize = viewportSize / 2;

    auto output = FRHITextureCreateDesc::Create2D(
       TEXT("GoL_State"),
       texSize,
       PF_R8G8 //Two-channel unorm, 8 bits per channel
    );
    output.AddFlags(TexCreate_ShaderResource | TexCreate_UAV |
                    TexCreate_RenderTargetable);

    return output;
}
FGameOfLifeView::FGameOfLifeView(FRDGBuilder& graph,
                                 const FViewInfo& view,
                                 const FIntRect& viewportSubset)
    : F_EGP_ViewPersistentData(graph, view, viewportSubset)
{
    auto desc = SimStateDesc(viewportSubset.Size());
    SimState = RHICreateTexture(desc);
    SimBuffer = RHICreateTexture(desc);
}
```

Next we need to handle what happens when a player's viewport gets resized. This is where we add our first, and simplest, shader.

```
//In FGameOfLifeView:
virtual void Resample(FRDGBuilder& graph, const FViewInfo& view,
                      const FInt32Point& oldResolution,
                      const FInt32Point& newResolution,
                      const FInt32Point& offsetDelta) override;

//In the cpp:
struct FGoLResamplePS : public FGlobalShader
{
    DECLARE_GLOBAL_SHADER(FGoLResamplePS);
    SHADER_USE_PARAMETER_STRUCT(FGoLResamplePS, FGlobalShader)

    //In practice most parameter structs are only used by one shader,
    //   so instead of a typedef you can directly define the struct
    //   within its shader class and name it FParameters.
    BEGIN_SHADER_PARAMETER_STRUCT(FParameters, )
```

```
        SHADER_PARAMETER_RDG_TEXTURE_SRV(Texture2D<float2>, InputTex)
        SHADER_PARAMETER_SAMPLER(SamplerState, InputSampler)
        RENDER_TARGET_BINDING_SLOTS()
    END_SHADER_PARAMETER_STRUCT()
};
IMPLEMENT_GLOBAL_SHADER(FGoLResamplePS, "/GameOfLife/Resample.usf",
                        "Main", SF_Pixel);
void FGameOfLifeView::Resample(FRDGBuilder& graph, const FViewInfo& view,
                               const FInt32Point& oldResolution,
                               const FInt32Point& newResolution,
                               const FInt32Point& offsetDelta)
{
    //The sim state texture doesn't share viewport space
    //    like the main render-targets do, so we don't care
    //    about position changes -- only resolution changes.
    if (oldResolution == newResolution)
        return;

    //Get an RDG handle to the old state.
    auto oldState = SimState;
    auto oldStateRDG = RegisterExternalTexture(
                            graph, oldState,
                            TEXT("Previous_GoL_State")
                        );

    //Allocate the new resized state, and get an RDG handle to it.
    auto newDesc = SimStateDesc({ newResolution.X, newResolution.Y });
    SimState = RHICreateTexture(newDesc);
    SimBuffer = RHICreateTexture(newDesc);
    auto newStateRDG = RegisterExternalTexture(
                             graph, SimState,
                             TEXT("Next_GoL_State")
                        );

    //NOTE: If the resolution change is more than 2x down or up
    //    then we will lose data; not a big deal for demo purposes
    //    but in real scenarios you may want to run several downscaling passes
    auto* params = graph.AllocParameters<FGoLResamplePS::FParameters>();
    params->InputTex = graph.CreateSRV(FRDGTextureSRVDesc{ oldStateRDG });
    params->InputSampler = TStaticSamplerState<SF_Bilinear, AM_Clamp, AM_Clamp>
    params->RenderTargets[0] = { newStateRDG, ERenderTargetLoadAction::ENoActio

    //Unreal's "draw screen pass" will by default:
    //    * Cover the whole screen using opaque blending
    //    * Use their own trivial Vertex Shader “FScreenPassVS”
    //This is perfect for our use-case.
    AddDrawScreenPass(
        graph, RDG_EVENT_NAME("ResizeGoLState"), view,
```

```
            FScreenPassTextureViewport{ newStateRDG },
            FScreenPassTextureViewport{ oldStateRDG },
            TShaderMapRef<FGoLResamplePS>{ view.ShaderMap },
            params
        );
    }
```

If you try starting Unreal now, you'll get an error that the shader file *Resample.usf* doesn't exist. Add it to your plugin's shader directory, *Shaders/Resample.usf*, containing the following code:

```
#include "/Engine/Private/Common.ush"

//Parameters:
Texture2D<float2> InputTex;
SamplerState InputSampler;

//Implementation:
void Main(float4 pixelPos4 : SV_Position,
          //Input from FScreenPassVS:
          noperspective float4 uvAndScreenPos : TEXCOORD0,
          //Output to the sim state render target:
          out float2 newState : SV_Target0)
{
    newState = InputTex.SampleLevel(InputSampler, uvAndScreenPos.xy, 0);
}
```

Finally, we need to use this data as part of our custom render pass object, `U_GOL_RenderPass`. Clear out its code from the previous article, which randomly cleared render targets, and add new code to track our sim state per-view.

```
//In U_GOL_RenderPass:
T_EGP_PerViewData<FGameOfLifeView> PerViewData;
virtual void Tick_RenderThread(const FSceneInterface& thisScene,
                               float gameThreadDeltaSeconds) override
{
    Super::Tick_RenderThread(thisScene, gameThreadDeltaSeconds);
```

```
    //Prune any viewport data that went unused for too many ticks.
    PerViewData.Tick();
}

//In U_GOL_RenderPass::PostRenderBasePassDeferred_RenderThread():
check(_view.bIsViewInfo);
auto& view = reinterpret_cast<FViewInfo&>(_view);
if (!Pass->ViewFilter->ShouldRenderFor(view))
    return;
//Get or create the GoL sim data for this view.
auto& simData = Pass->PerViewData.DataForView(graph, view);
//Soon we will draw simData to the screen right here
```

Sim State Initialization

Now we have sim data, but no initial values for the sim state. We'll initialize it using Perlin-esque noise to decide whether each cell starts Alive (1) or Dead (0). This requires a new shader.

Add this code to your cpp file:

```
struct FGoLInitializePS : public FGlobalShader
{
    DECLARE_GLOBAL_SHADER(FGoLInitializePS);

    BEGIN_SHADER_PARAMETER_STRUCT(FParameters, )
       SHADER_PARAMETER(float, Seed)
       SHADER_PARAMETER(float, NoiseScale)
       RENDER_TARGET_BINDING_SLOTS()
    END_SHADER_PARAMETER_STRUCT()
    SHADER_USE_PARAMETER_STRUCT(FGoLInitializePS, FGlobalShader)
};
IMPLEMENT_GLOBAL_SHADER(FGoLInitializePS, "/GameOfLife/Init.usf", "Main", SF_Pi
```

and this HLSL code to *Shaders/Init.usf*:

```
#include "/Engine/Private/Common.ush"

//Parameters:
```

```
    float Seed;
    float NoiseScale;

    //Implementation:
    void Main(float4 pixelPos4 : SV_Position,
              //Input from FScreenPassVS:
              noperspective float4 uvAndScreenPos : TEXCOORD0,
              //Output to the sim state render target:
              out float2 newState : SV_Target0)
    {
        //Unreal's 'Random.ush' gives us a lot of nice PRNG math,
        //    and has already been included through 'Common.ush'.
        #define GOL_NOISE(scale, seed2) \
           GradientNoise3D_ALU(float3( \
               uvAndScreenPos.xy * NoiseScale * scale, \
               Seed + seed2 \
              ), false, 1.0)

        //Generate a random initial state.
        newState.y = (0.75 * GOL_NOISE(1.0, 0.0)) +
                     (0.20 * GOL_NOISE(2.0, 2.456)) +
                     (0.05 * GOL_NOISE(4.0, 1.8223));

        //The first channel is supposed to be a discrete bool.
        newState.x = step(0.5, newState.y);
    }
```

Before executing this shader, we need values for its parameters. Go to the pass object (`U_GOL_RenderPass`) and add some user-facing parameters:

```
    //Seed that's used when initializing the sim for a new viewport.
    UPROPERTY(BlueprintReadWrite, EditAnywhere)
    float NewViewportSeed = 1.432;

    //The size of the sim's initial noise for a new viewport.
    UPROPERTY(BlueprintReadWrite, EditAnywhere)
    float NewViewportNoiseScale = 10.0f;
```

Were these any more complex than a float, we'd add Tick logic that safely uploads them into render-thread-only copies to prevent race conditions. That will happen

with other parameters later on.

There is also normally a risk that the pass or subsystem gets destroyed in the game thread while its SVE is still trying to read these parameters on the render thread. However my library ensures this won't happen, using the fence logic covered in part 2.

Finally, with all these pieces in place, we can add logic to run the initialization shader on every new viewport. Modify the constructor for `FGameOfLifeView` to take the parameters and invoke the shader:

```
FGameOfLifeView::FGameOfLifeView(FRDGBuilder& graph,
                                 const FViewInfo& view,
                                 const FIntRect& viewportSubset,
                                 float seed, float noiseScale)
    : F_EGP_ViewPersistentData(graph, view, viewportSubset)
{
    auto desc = SimStateDesc(viewportSubset.Size());
    SimState = RHICreateTexture(desc);
    SimBuffer = RHICreateTexture(desc);

    //Initialize the sim state using our pixel shader.
    auto simStateRDG = RegisterExternalTexture(graph, SimState,
                                               TEXT("GoL_InitialState"));
    auto* initShaderParams = graph.AllocParameters<FGoLInitializePS::FParameter
    initShaderParams->Seed = seed;
    initShaderParams->NoiseScale = noiseScale;
```

```
        initShaderParams->RenderTargets[0] = {
            simStateRDG,
            //Previous RT contents are not needed
            ERenderTargetLoadAction::ENoAction
        };
        AddDrawScreenPass(
            graph, RDG_EVENT_NAME("GoL_Initialize"), view,
            FScreenPassTextureViewport{ simStateRDG },
            FScreenPassTextureViewport{ simStateRDG->Desc.Extent },
            TShaderMapRef<FGoLInitializePS>{ view.ShaderMap },
            initShaderParams
        );
    }
```

These extra two constructor parameters must be passed in when we try to retrieve the view, in the SVE:

```
//In U_GOL_RenderPass::PostRenderBasePassDeferred_RenderThread():
auto& viewData = Pass->PerViewData.DataForView(
    graph, view,
    //If this is the first time encountering this view,
    //    the below arguments go into its constructor.
    //Dereferencing the Pass object within its SVE is OK!
    //Using game-thread-accessible data from the render-thread is also OK
    //    because they're just cosmetic floats!
    Pass->NewViewportSeed, Pass->NewViewportNoiseScale
);
```

With this in place, your project should automatically create a randomized Game of Life sim for each viewport in the running game. However, there's no way to see this texture yet, and no logic to update it every frame.

Display Shader

Drawing the simulation on top of the screen could probably be accomplished with a normal Post-Process Material, but only with some annoying scripting to feed the sim state into the Material at runtime. It would also be tricky to support the effect appearing in more than one viewport. Finally, we couldn't add the sim at more interesting stages such as drawing into the albedo right before lighting is calculated.

Instead we are going to set up our SVE to execute this new pass, just before the post-processing phase, writing to the Scene Color output. To start we need to update our module's config, to include some header files related to post-processing.

```
//In the module's .Build.cs file:
PrivateIncludePaths.AddRange(new string[] {
    System.IO.Path.Combine(GetModuleDirectory("Renderer"), "Private"),
});
```

Now let's define the shader's class in the cpp, with a helper function to invoke it:

```
struct FGoLDisplayPS : public FGlobalShader
{
    DECLARE_GLOBAL_SHADER(FGoLDisplayPS);

    BEGIN_SHADER_PARAMETER_STRUCT(FParameters, )
       SHADER_PARAMETER_RDG_TEXTURE_SRV(Texture2D<float2>, SimState)
       SHADER_PARAMETER_SAMPLER(SamplerState, SimSampler)
       //To calculate UV in 0-1 space rather than the viewport's sub-set:
       SHADER_PARAMETER(FUint32Vector4, ViewportMinAndSize)
       RENDER_TARGET_BINDING_SLOTS()
    END_SHADER_PARAMETER_STRUCT()
    SHADER_USE_PARAMETER_STRUCT(FGoLDisplayPS, FGlobalShader)
};

IMPLEMENT_GLOBAL_SHADER(FGoLDisplayPS, "/GameOfLife/Display.usf", "Main", SF_Pi

//Helper function to assemble the GoL draw call.
static void RenderGoLState(FRDGBuilder& graph, const FViewInfo& view,
                           FRDGTextureRef simStateTex,
                           FRHIBlendState* blending,
                           const FRenderTargetBinding& output)
{
    auto* params = graph.AllocParameters<FGoLDisplayPS::FParameters>();
    params->SimState = graph.CreateSRV(FRDGTextureSRVDesc{ simStateTex });
    params->SimSampler = TStaticSamplerState<SF_Bilinear, AM_Clamp, AM_Clamp>::
    params->ViewportMinAndSize = FUint32Vector4(
       view.ViewRect.Min.X, view.ViewRect.Min.Y,
       view.ViewRect.Width(), view.ViewRect.Height()
```

```
    );
    params->RenderTargets[0] = output;

    TShaderMapRef<FGoLDisplayPS> pixelShader{ view.ShaderMap };

    AddDrawScreenPass(
       graph, RDG_EVENT_NAME("GoL_Display"), view,
       FScreenPassTextureViewport{ output.GetTexture() },
       FScreenPassTextureViewport{ simStateTex },
       //The overload that lets us pass a blend mode
       //   also requires that we pass a Vertex Shader.
       //This is the usual built-in screen pass VS.
       TShaderMapRef<FScreenPassVS>{ view.ShaderMap },
       pixelShader, blending, params
    );
}
```

Next we'll add the shader code, in *Shaders/Display.usf*.

```
#include "/Engine/Private/Common.ush"

//Uniforms:
Texture2D<float2> SimState;
SamplerState SimSampler;
uint4 ViewportMinAndSize;

//Implementation:
float4 Main(in noperspective float4 uvAndScreenPos : TEXCOORD0,
            float4 pixelPos : SV_Position
           ) : SV_Target0
{
    //The UV from the vertex shader only covers a subset of the screen,
    //    for example in split-screen games.
    //But our sim state texture only contains pixels in that subset,
    //    so we want to use the full 0-1 UV space.
    float2 uv = (pixelPos.xy - float2(ViewportMinAndSize.xy)) /
                float2(ViewportMinAndSize.zw);

    float2 currentState = SimState.SampleLevel(SimSampler, uv, 0);

    //As mentioned before there is an official discrete value in X
    //    and a continuous value in Y.
    //Create visual effects from both these values and mix together.
    float discreteEffectStrength = 0.3,
```

```
            totalEffectStrength = pow(max(currentState.x, currentState.y),
                                      0.3);
        float3 discreteEffect = currentState.x * float3(1, 0, 0),
               continuousEffect = pow(currentState.y, float3(3.0, 1.75, 1.0)),
               totalEffect = lerp(continuousEffect, discreteEffect, discreteEffectS
        return float4(
            lerp(1.0, totalEffect, totalEffectStrength),
            1
        );
    }
```

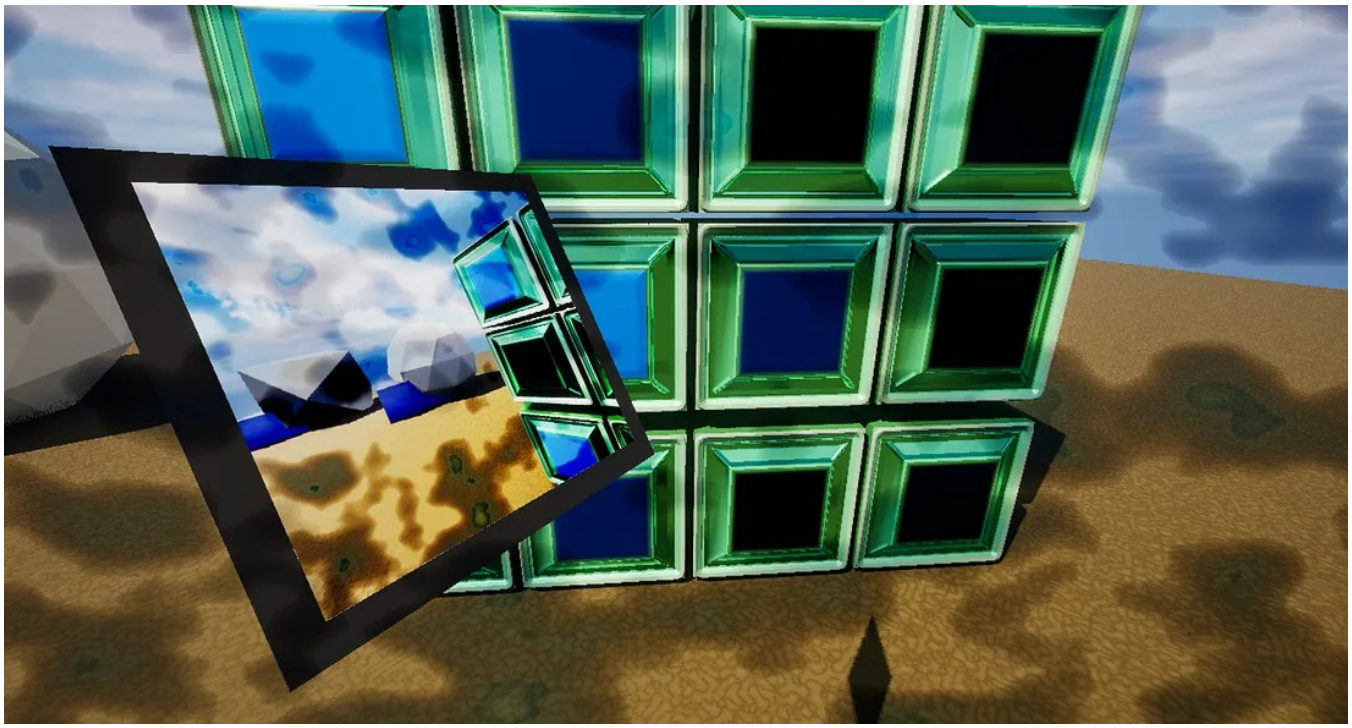
You may be tempted to play with the display shader, and by all means — you can tweak the file then press "**Ctrl+Shift+.**" in the editor to quickly recompile it. But don't forget that it gets much easier to play with after we convert these to Material shaders in the next article!

Finally we need to update the SVE to invoke the shader. Replace its "post-deferred" virtual function with the "pre-postprocess" one:

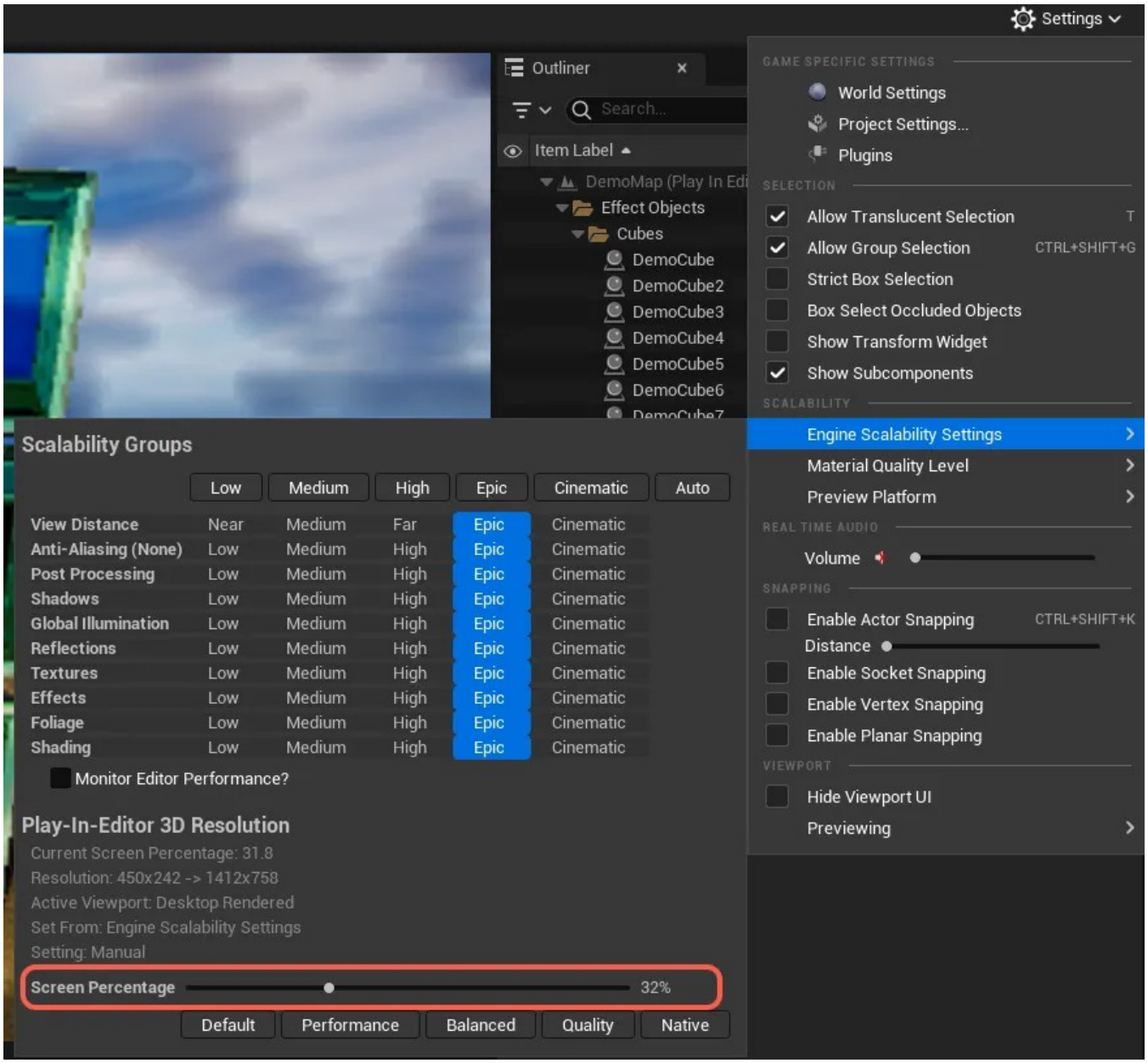
```
void F_GOL_PassSVE::PrePostProcessPass_RenderThread(FRDGBuilder& graph,
                                                    const FSceneView& _view,
                                                    const FPostProcessingInputs
{
    check(_view.bIsViewInfo);
    auto& view = reinterpret_cast<const FViewInfo&>(_view);

    //Get or create the per-view data.
    auto& viewData = Pass->PerViewData.DataForView(
       graph, view,
       Pass->NewViewportSeed, Pass->NewViewportNoiseScale
    );

    //Draw the sim onto the scene color texture.
    auto simStateRDG = RegisterExternalTexture(graph, viewData.SimState,
                                               TEXT("GoL_State"));
    RenderGoLState(
       graph, view, simStateRDG,
       //Use Multiply blending.
       TStaticBlendState<CW_RGBA, BO_Add, BF_DestColor, BF_Zero>::GetRHI(),
       {
           inputs.SceneTextures->GetContents()->SceneColorTexture,
           //We're blending on top of the current scene color,
```

```
            //    so make sure its existing contents are Loaded when bound.
            ERenderTargetLoadAction::ELoad
        }
    );
}
```

With all of this written, you should be able to see a cloudy noise overlaid on the screen when you press Play.



You should also change your screen percentage during play to verify that the Resample shader works as expected.



Be aware, you can only change the percentage during PIE for some reason.

Simulation

The fourth and final shader will run one iteration of the Game of Life on our viewports' state textures. Don't worry if you don't know the actual algorithm; the math isn't as important as the general idea of making an animated, persistent render effect.

In the header, define some sim parameters.

```
USTRUCT(BlueprintType)
struct GOL_DEMO_API FGameOfLifeSettings
{
  GENERATED_BODY()
public:

  UPROPERTY(BlueprintReadWrite, EditAnywhere, meta=(ClampMin=0))
  float OverallSpeed = 2.0f;
  UPROPERTY(BlueprintReadWrite, EditAnywhere, meta=(ClampMin=0))
  float MinSpeed = 0.3f;
  UPROPERTY(BlueprintReadWrite, EditAnywhere, meta=(ClampMin=0))
  float AccelerationExponent = 1.0f;
};
```

In your pass object, provide those parameters to the user.

```
UPROPERTY(BlueprintReadWrite, EditAnywhere)
FGameOfLifeSettings SimSettings;
```

In your cpp file, set up and invoke a Simulate shader.

```
struct FGoLSimulatePS : public FGlobalShader
{
  DECLARE_GLOBAL_SHADER(FGoLSimulatePS);

  BEGIN_SHADER_PARAMETER_STRUCT(FParameters, )
    SHADER_PARAMETER_RDG_TEXTURE_SRV(Texture2D<float2>, PreviousState)
    SHADER_PARAMETER(float, DeltaSeconds)

    //Sim control parameters:
    SHADER_PARAMETER(float, OverallSpeed)
    SHADER_PARAMETER(float, MinSpeed)
    SHADER_PARAMETER(float, AccelerationExponent)

    RENDER_TARGET_BINDING_SLOTS()
  END_SHADER_PARAMETER_STRUCT()
```

```
  SHADER_USE_PARAMETER_STRUCT(FGoLSimulatePS, FGlobalShader)
};
IMPLEMENT_GLOBAL_SHADER(FGoLSimulatePS, "/GameOfLife/Simulate.usf", "Main", SF_
```

```
static void UpdateGoLState(FRDGBuilder& graph,
                           ERHIFeatureLevel::Type featureLevel,
                           FRDGTextureRef currentSimState,
                           FRDGTextureRef nextSimState,
                           float deltaSeconds,
                           const FGameOfLifeSettings& settings)
{
  //The shader assumes input and output are the same size.
  check(currentSimState->Desc.Extent == nextSimState->Desc.Extent);

  auto* params = graph.AllocParameters<FGoLSimulatePS::FParameters>();
  params->PreviousState = graph.CreateSRV(FRDGTextureSRVDesc{ currentSimState }
  params->DeltaSeconds = deltaSeconds;
  params->OverallSpeed = settings.OverallSpeed;
  params->MinSpeed = settings.MinSpeed;
  params->AccelerationExponent = settings.AccelerationExponent;
  params->RenderTargets[0] = { nextSimState, ERenderTargetLoadAction::ENoAction

  auto* shaderMap = GetGlobalShaderMap(featureLevel);
  AddDrawScreenPass(
    graph, RDG_EVENT_NAME("GoL_Tick"), featureLevel,
    FScreenPassTextureViewport{ nextSimState },
    FScreenPassTextureViewport{ nextSimState->Desc.Extent },
    TShaderMapRef<FGoLSimulatePS>{ shaderMap },
    params
  );
}
```

In a new file *Shaders/Simulate.usf*, add the shader code.

```
#include "/Engine/Private/Common.ush"

//Basic parameters
float DeltaSeconds;
Texture2D<float2> PreviousState;

//User-controlled parameters
```

```
float OverallSpeed;
float MinSpeed;
float AccelerationExponent;

//The Main function directly returns the new state.
float2 Main(in noperspective float4 uvAndScreenPos : TEXCOORD0,
            float4 pixelPos : SV_Position
           ) : SV_Target0
{
  uint2 stateSize;
  PreviousState.GetDimensions(stateSize.x, stateSize.y);

  //Sample the neighboring cell states.
  float2 prevStates[9];
  int2 ourPixel = int2(pixelPos.xy);
  const int ourIdx = 4;
  for (int x = -1; x <= 1; ++x)
    for (int y = -1; y <= 1; ++y)
    {
      int2 globalIdx = ourPixel + int2(x, y);
      int localIdx = (x + 1) + ((y + 1) * 3);
      bool inbounds = all(bool4(globalIdx >= 0,
                                globalIdx < stateSize);
      prevStates[localIdx] = inBounds ?
        PreviousState[globalIdx] :
        float2(0, 0);
    }

  //Count the number of living neighbors.
  float neighborStrength = 0;
  const float maxStrength = 8.0;
  for (int i = 0; i < 9; ++i)
    if (i != ourIdx)
      neighborStrength += prevStates[i].x;

  //Apply the usual Game of Life rules to our cell,
  //  dying/living/resurrecting based on living neighbor count.
  //Also calculate a "severity" estimating the strength
  //  of the pull towards the new state.
  float targetValue, severityT, speedScale;
  const float ThresholdTooFew = 2,
              ThresholdResurrect = 2.5,
              ThresholdTooMany = 3;
  if (neighborStrength < ThresholdTooFew)
  {
    //Die off, from underpopulation.
    targetValue = 0;
    severityT = 1.0 - (neighborStrength / ThresholdTooFew);
```

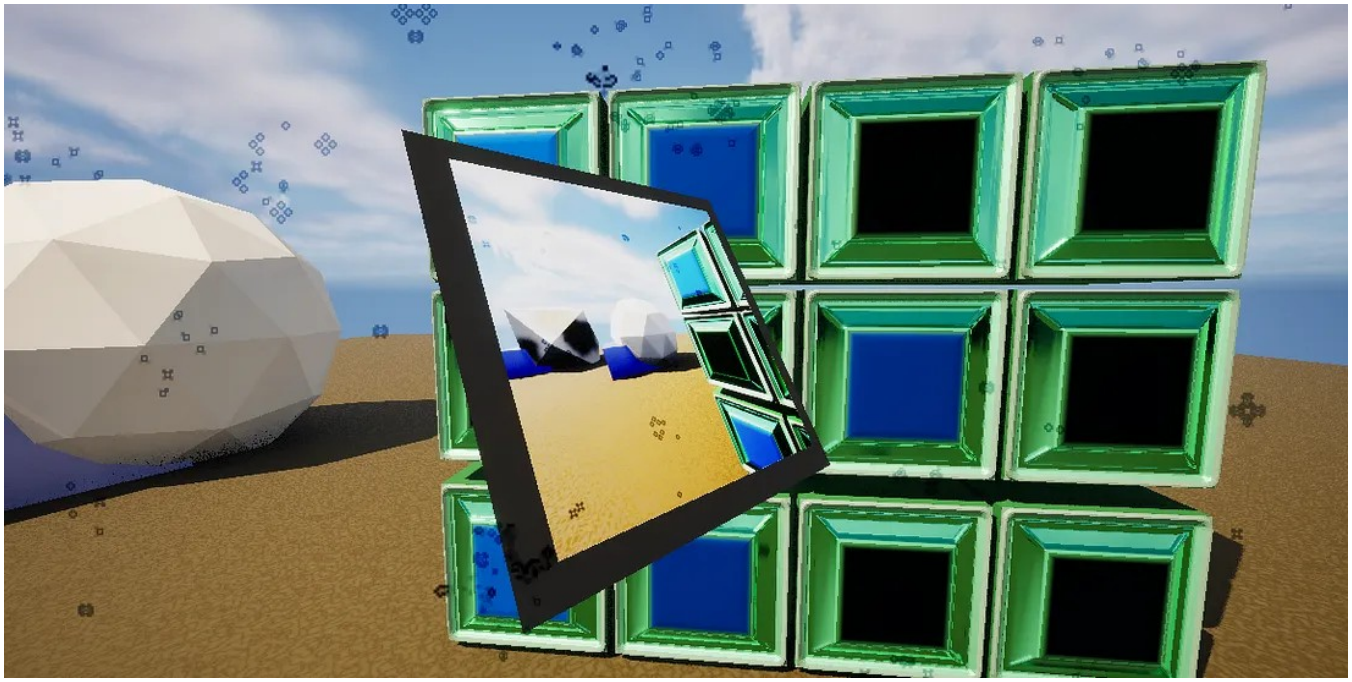
```
    speedScale = 1.0;
  }
  else if (neighborStrength < ThresholdResurrect)
  {
    //Maintain current living status, from a normal population.
    targetValue = prevStates[ourIdx].x;
    severityT = 1.0;
    speedScale = 1.0;
  }
  else if (neighborStrength <= ThresholdTooMany)
  {
    //Come alive, from an ideal population.
    targetValue = 1;
    severityT = (neighborStrength - ThresholdResurrect) /
                  max(0.0001, ThresholdTooMany - ThresholdResurrect);
    speedScale = 1.0;
  }
  else
  {
    //Die, from overpopulation.
    targetValue = 0;
    severityT = (neighborStrength - ThresholdTooMany) /
                  max(0.0001, maxStrength - ThresholdTooMany);
    speedScale = 1.0;
  }
  severityT = pow(saturate(severityT),
                  1.0 / max(0.00001, AccelerationExponent));

  //Now that we know our new discrete state,
  //    update our continuous state accordingly.
  float smoothValue = prevStates[ourIdx].y;
  float frameSpeed = DeltaSeconds *
                     max(MinSpeed, speedScale * severityT * OverallSpeed),
        frameVelocity = frameSpeed * sign(targetValue - smoothValue);
  if (distance(smoothValue, targetValue) < frameSpeed)
    smoothValue = targetValue;
  else
    smoothValue += frameVelocity;

  return float2(targetValue, smoothValue);
}
```

In our render pass object, execute the sim tick for each viewport.

```
void U_GOL_RenderPass::Tick_RenderThread(const FSceneInterface& thisScene, floa
{
  //These two lines should already exist
  Super::Tick_RenderThread(thisScene, gameThreadDeltaSeconds);
  PerViewData.Tick();

  //Run our "simulate" shader on all viewports.
  FRDGBuilder graph{ FRHICommandListImmediate::Get(),
                     RDG_EVENT_NAME("UpdateAllGoLViewports") };
  //the sim settings struct is POD, and its values are cosmetic,
  //    so there is no need to avoid race conditions
  //    by keeping a separate render-thread copy.
  auto simSettings = SimSettings;
  PerViewData.ForEachView([&](int viewID, FGameOfLifeView& view,
                              ERHIFeatureLevel::Type featureLevel) {
    UpdateGoLState(
      graph, featureLevel,
      RegisterExternalTexture(graph, view.SimState,
                              TEXT("GoL_PrevState")),
      RegisterExternalTexture(graph, view.SimBuffer,
                              TEXT("GoL_NextState")),
      gameThreadDeltaSeconds,
      simSettings
    );
    std::swap(view.SimBuffer, view.SimState);
  });
  graph.Execute();
}
```

That's it! Now the Game of Life simulation should run as soon as you start running the game, and look like the animation at the top of this Demo section.

A screenshot after some shader and parameter tweaks



Each viewport gets its own effect, subject to the pass object's **ViewFilter**

If you add the following line in your pass's Init function, you can see that the effect only applies to the player's view and not to Scene Capture Components:

```
ViewFilter->FilterByPlayerIdx(0);
```

Code

The global shaders are added to the demo in the *part/4* branch:

heyx3/UnrealExtendedGraphicsProgrammingDemo at part/4

Details

Permutations

Shader permutations are created by defining special helper structs that represent each one, then shadowing the typedef `FPermutationDomain` to list those helper structs. For example:

```
//Inside the shader class:

//The shader will either see
//    '#define PARALLAX_MAPPING 0', or '#define PARALLAX_MAPPING 1'.
class FEnableParallaxMapping : SHADER_PERMUTATION_BOOL("PARALLAX_MAPPING");
//The shader will see '#define ITER_COUNT n' where n is 0, 1, or 2.
class FIterationCount : SHADER_PERMUTATION_INT("ITER_COUNT", 3);

using FPermutationDomain = TShaderPermutationDomain<
    FEnableParallaxMapping,
    FIterationCount
>;
```

An instance of `FPermutationDomain` can be converted to a unique ID and back again using `domain.ToDimensionValueId()`. Generally functions which take a permutation or permutation ID will default to 0 for shaders that don't have any permutations.

To retrieve a specific permutation from a shader map, do this:

```
FMyShaderPS::FPermutationDomain permutation;
permutation.Set<FMyShaderPS::FEnableParallaxMapping>(true);
permutation.Set<FMyShaderPS::FIterationCount>(1);
TShaderMapRef<FMyShaderPS> myShader{ view.ShaderMap, permutation };
```

Once a shader has multiple permutation parameters it's highly recommended to eliminate redundant ones by shadowing the static function `RemapPermutation(...)`. For example, if permutation integer *A* only matters when permutation bool *B* is true, then `RemapPermutation(...)` should set *A* to a constant value when *B* is false. Anybody who's made or played Unreal games knows how painful shader permutation compiling can be!

The Shader Class

Conventionally, shader classes are suffixed based on the kind of shader they are — VS, PS, CS, etc.

Some other static shader functions you can shadow are:

- **`static bool ShouldCompilePermutation(const FGlobalShaderPermutationParameters& permutation)`** decides whether your shader should be compiled in a particular environment. For example your shader may require the SM6 shader model, or the Metal graphics API, or editor-only data. If you have custom shader permutations, then you could also check them here to reduce the number of shaders in your game.
- **`static void ModifyCompilationEnvironment(const FShaderPermutationParameters& permutation, FShaderCompilerEnvironment& env)`** can customize the compilation environment for your shader. Add virtual include paths, *#define* preprocessor tokens, enable specific compiler flags (see *ECompilerFlags*), and more.
- **`static bool ValidateCompiledResult(EShaderPlatform platform, const FShaderParameterMap& parameters, TArray<FString>& outErrorMessages)`** lets you return false and generate error messages if something is wrong with your

shader's inputs and outputs, for example if it tries to read from an input that you know won't exist in the places you invoke it. This one is very situational.

The empty parameters you see in shader class macros are for one of two purposes:

- Template arguments, in case your shader class is templated.
- Your DLL export statement (like `MYMODULE_API`), in case you want the shader to be used by outsiders. If you don't want that then it's recommended to keep the shader entirely inside a cpp file.

It seems possible to write Global or Material shaders that do simple 3D scene drawing, instead of the much more complex Mesh-Material shaders which draw the "mesh batches" contained in drawable components. For in-engine examples of simple 3D rendering see `FDeferredDecalVS` (a global shader) and its partner `FDeferredDecalPS` (a Material shader).

Legacy shader parameters (the alternative to parameter structs)

This is how to define and upload shader parameters without the use of an `FParameters` struct.

Fields

When you used a parameter struct, its fields were being automatically bound through the inherited field `FShader::Bindings`. Now we will supplant this with individual binding fields for each parameter. All shader parameters must be declared as a field, in one of the following ways based on their type:

```
LAYOUT_FIELD(FShaderParameter, MyNormalParam);
LAYOUT_FIELD(FShaderResourceParameter, MySrvOrUavOrSamplerParam);
LAYOUT_FIELD(FShaderUniformBufferParameter, MyNestedParamStruct);
LAYOUT_FIELD(FShaderUniformBufferParameter, MyGlobalBufferRef);
```

Note that `FShaderUniformBufferParameter` can represent both persistent global uniform buffers, and local/short-term ones (effectively pasting in the contents of a parameter

struct). There is a slight difference in how each gets uploaded, mentioned below.

Next define constructors for your shader that tie those fields to HLSL parameters. With parameter structs these constructors had been generated for you, inside `SHADER_USE_PARAMETER_STRUCT`. When binding parameters you should describe them as “optional” or “mandatory”, describing whether it’s an error for the parameter to be missing from the shader.

```
FMyShaderPS() = default;
FMyShaderPS(const ShaderMetaType::CompiledShaderInitializerType& initializer)
    : FGlobalShader(initializer) //Or whatever your parent type is
{
    MyNormalParam.Bind(
        initializer.ParameterMap,
        TEXT(“MyFloat”), //The name in HLSL
        SPF_Mandatory //We know the float exists in our shader
    );
    MySrvOrUavOrSamplerParam.Bind(
        initializer.ParameterMap,
        TEXT(“MyOutputTex”), //The name in HLSL
        SPF_Mandatory //The output of this shader is important!
    );
    MyNestedParamStruct.Bind(
        Initializer.ParameterMap,
        //Get the HLSL name of the buffer
        FSomeParamStruct::FTypeInfo::GetStructMetadata()
            ->GetShaderVariableName()
    );
    MyGlobalUniformBuffer.Bind(
        Initializer.ParameterMap,
        //Get the HLSL name of the buffer
        FSomeGlobalUniformBuffer::FTypeInfo::GetStructMetadata()
            ->GetShaderVariableName()
    );
}
```

Upload

TL;DR The basic workflow for dispatching a compute shader using legacy parameter fields should look like below. Other shader types work very similarly, just with more extraneous

details like binding vertex/index buffers. The interesting part for us is lines 2 and 4, which you can read through to see how they unpack boilerplate and interact with your shader class.

```
SetComputePipelineState(cmdList, shaderRef.GetComputeShader());
SetShaderParametersLegacyCS(cmdList, shaderRef, params…)
DispatchComputeShader(cmdList, shaderRef,
                      groupSizeX, groupSizeY, groupSizeZ);
UnsetShaderParametersLegacyCS(cmdList, shaderRef);
```

To store the parameters for upload, grab a memory block from the RHI command-list with

```
auto& parameterMemory = cmds.GetScratchShaderParameters();
```

Call global functions which write into this memory — primarily `SetShaderValue(…)`, `SetShaderValueArray(…)`, `SetSRVParameter(…)`, `SetUAVParameter(…)`, and `SetUniformBufferParameter[Immediate](…)`. These functions will take your memory block, one of your binding fields defined above, and the new value for that field.

After that's done, upload all your parameters to your shader on the GPU with

```
cmds.SetBatchedShaderParameters(shaderRHIHandle, parameterMemory)
```

Now you can execute your shader!

NOTE: *`SetUniformBufferParameterImmediate` is for one-off buffer structs that you allocated for this draw call, while `SetUniformBufferParameter` is for persistent/global buffers. In some cases the engine doesn't appear to explicitly call `SetUniformBufferParameter`, and I haven't figured out why yet…perhaps the RDG steps in*

to handle it for you?

***NOTE:** With both kinds of uniform buffers (immediate and persistent), rather than taking the binding field directly, Unreal code usually gets the binding with `myShader->GetUniformBufferParameter<FMyBufferStruct>()`. I don't know why, or what happens if you pass the binding field directly instead.*

When you need to upload params to more than one shader (e.g. VS and PS), you can reuse the parameter memory block. Just make sure to call `parameterMemory.Reset()` in-between them. This is already called automatically each time you retrieve the memory block from the command-list.

Implement `SetParameters()`

To remove boilerplate and chances for error, there is a common convention to keep the parameter fields private and define a public member function named `SetParameters` which sets all of them at once. Then you can call the global functions `SetShaderParametersLegacy[PS|VS|CS|etc](cmdList, shaderRef, args…)` which handles most of the boilerplate and offers a very simplified and safe interface for configuring your shader. This is especially valuable if your shader is exposed in a header for others to use.

***NOTE:** Some engine shader classes also have an old deprecated `SetParameters` implementation with a different signature; ignore those in favor of the convention described above.*

Finally, after execution it's important to unbind all your SRV and UAV resources. Fortunately the syntax is extremely similar to binding them: get a memory block with

```
auto& unbindingMemory = cmds.GetScratchShaderUnbinds();
```

and make calls to `UnsetSRVParameter(…)` and `UnsetUAVParameter(…)`, then finalize it

with

```
cmds.SetBatchedShaderUnbinds(shaderRHIHandle, unbindingMemory);
```

Just like the convention of defining SetParameters and calling `SetShaderParametersLegacy[PS|VS|CS|etc]`, there is an identical convention for unbinding resources: define `UnsetParameters(…)` for your class and then you can call `UnsetShaderParametersLegacy[PS|VS|CS|etc]`.

NOTE: *It seems there are places in the engine which use legacy parameter code, but don't unset their SRV's and UAV's afterwards. Perhaps because they're inside an RDG pass which is already aware of those resources? More research is needed.*

Notably, the `SetParameters(…)` convention is not used for Mesh-Material shaders. These have a slightly different workflow involving per-primitive "elements" which hold your parameters.

Unreal Engine 5 | Graphics Programming | Shaders | Technical Art

Edit profile

Written by William Mishra-Manning

59 followers · 5 following

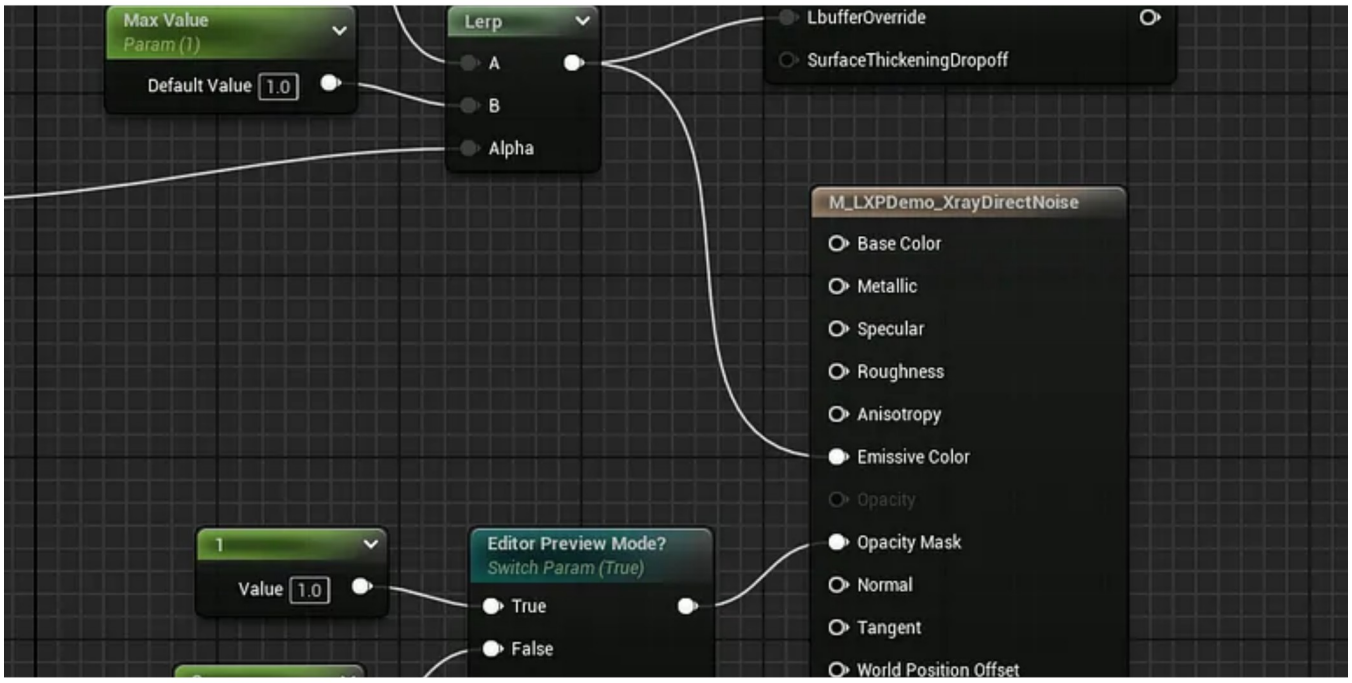
Graphics Programming, Procedural Generation

No responses yet

William Mishra-Manning

What are your thoughts?

More from William Mishra-Manning

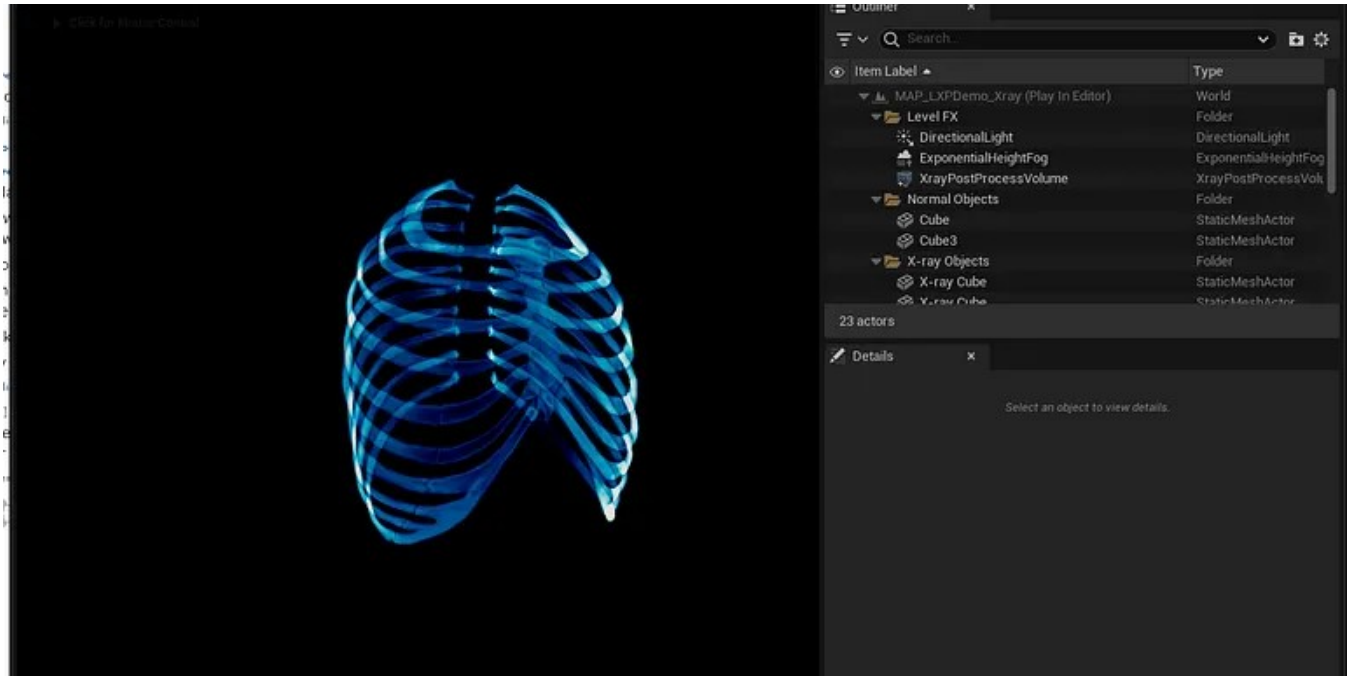


William Mishra-Manning

Advanced Graphics Programming in Unreal, part 6

This is a seven-part article series.

Jun 25, 2025 1 2

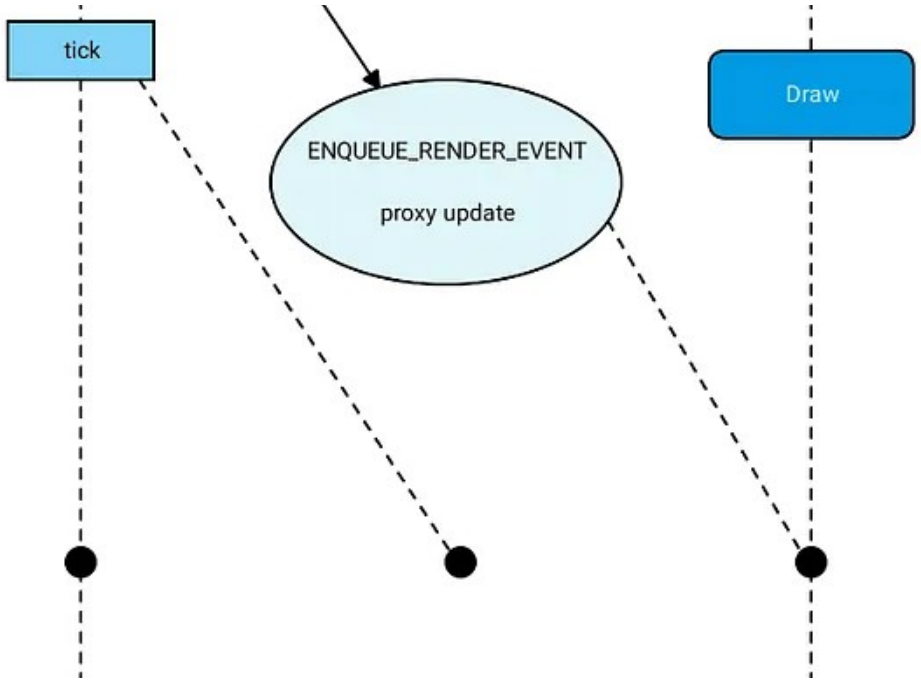


William Mishra-Manning

Advanced Graphics Programming in Unreal, part 1

This is a seven-part article series.

Jun 24, 2025 31 1



William Mishra-Manning

Advanced Graphics Programming in Unreal, part 2

This is a seven-part article series.

16 1

William Mishra-Manning

Advanced Graphics Programming in Unreal, part 3

This is a seven-part article series.

Jun 24, 2025 6

See all from William Mishra-Manning

Recommended from Medium

AmirHossein Aghajari

Liquid Glass: iOS Effect Explanation

Apple's Liquid Glass effect with shaders: math, distortion & chromatic aberration

Nov 23, 2025 240 1

Nikhil Maurya

RDG in Practice: From Shader Parameters to a Working Pass

The Render Dependency Graph (RDG) is Unreal Engine's way of describing GPU work : passes, resources and lifetimes. So the renderer can...

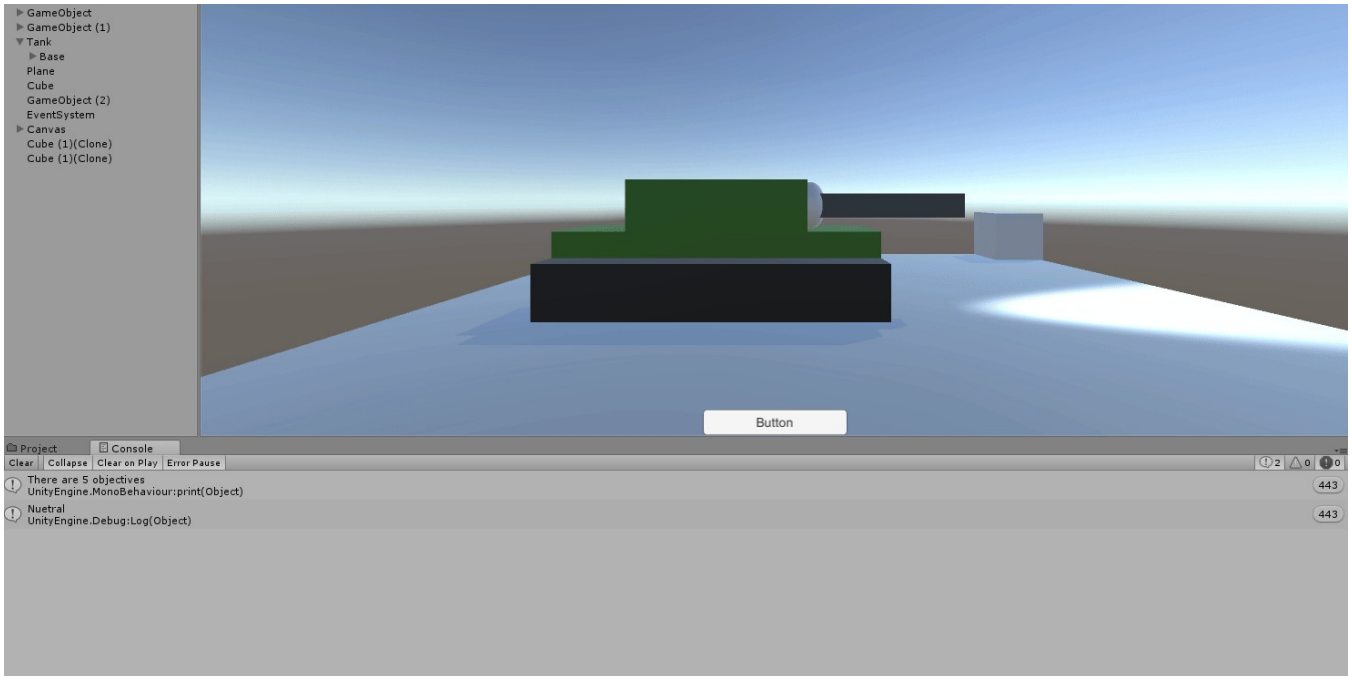
4

Akash Kadam

Setting Up FreeRTOS on STM32 / ESP32 from Zero A Hands-On Beginner's Guide

If you've already understood what FreeRTOS is and why bare-metal eventually breaks, the next natural question is:

Jan 29 1



Zac Bogner

Switching from Unity to Unreal

In the previous article, I wrote about Switch Statements in Unity.

Dec 22, 2025 3

1. **Start with Bayes' Rule:**

$$P(x|z) = \frac{P(z|x)P(x)}{P(z)}$$

2. **Divide by the "Free" state ($\neg x$) to create an Odds Ratio:**

$$\frac{P(x|z)}{P(\neg x|z)} = \frac{P(z|x)}{P(z|\neg x)} \cdot \frac{P(x)}{P(\neg x)}$$

3. **The Result:** This allows the robot to update its map by comparing how likely a measurement is if a wall exists ($P(z|x)$) versus if it doesn't ($P(z|\neg x)$).

Naren Suri

Occupancy Grid Mapping Algorithm

Occupancy Grid Mapping serves as a foundational framework in robotic perception, enabling a mobile agent to represent an unstructured...

1

In Towards Dev by Sagar

Using std::layout_stride with std::mdspan for Arbitrary Data Layouts in C++23

Beyond row-major: How to map multidimensional views to any memory structure using C++23's most flexible layout policy

Apr 4 6

See more recommendations